

LangChain Basic

LangChain 기초

01 LangChain 소개

목차 01 LangChain 소개

1. LangChain 이란
2. LangChain 주요 특징
3. LangChain과 API

1. LangChain 이란



거대 언어 모델(LLM)을 사용하여 애플리케이션을 개발하기 위한 오픈소스 프레임워크

앵무새와 사슬의 등장

- 2022년 10월, OpenAI의 ChatGPT의 등장으로 LLM 열풍이 일어났을 때 등장
- 개발자 해리슨 체이스(Harrison Chase)가 오픈소스로 공개되어 커뮤니티 구성원에 의해 발전하게 됨
- 앵무새는 언어 모델을 상징, 사슬(Chain)은 언어 모델과 언어 모델을 활용할 수 있는 다양한 도구를 결합시킨다는 의미



LangChain



LangChain의 등장 배경

LLM 단독 사용

GPT, Claude 등의
LLM은 강력한 생성
능력을 갖추

한계

그러나 단독 사용 시
일회성 응답,
문맥 지속 불가,
외부 지식 연결 부족 등
한계 존재

LangChain 등장

LLM을 **통합**하고,
체인화하며,
상태를 유지할 수 있는
구조가 필요

→ **LangChain 등장**



실용적인 애플리케이션 구현에 제약이 있음

프롬프트 설계의 복잡성

- 재사용 불가
- 일관성 부족

상태 비저장 (Stateless)

이전 대화나 정보 기억 불가

외부 데이터 접근 불가

- 사전 학습된 정보만 사용 가능
- 실시간 정보 활용 불가

기능 확장 어려움

계산, 검색, 요약 등의 작업 결합이 비효율적

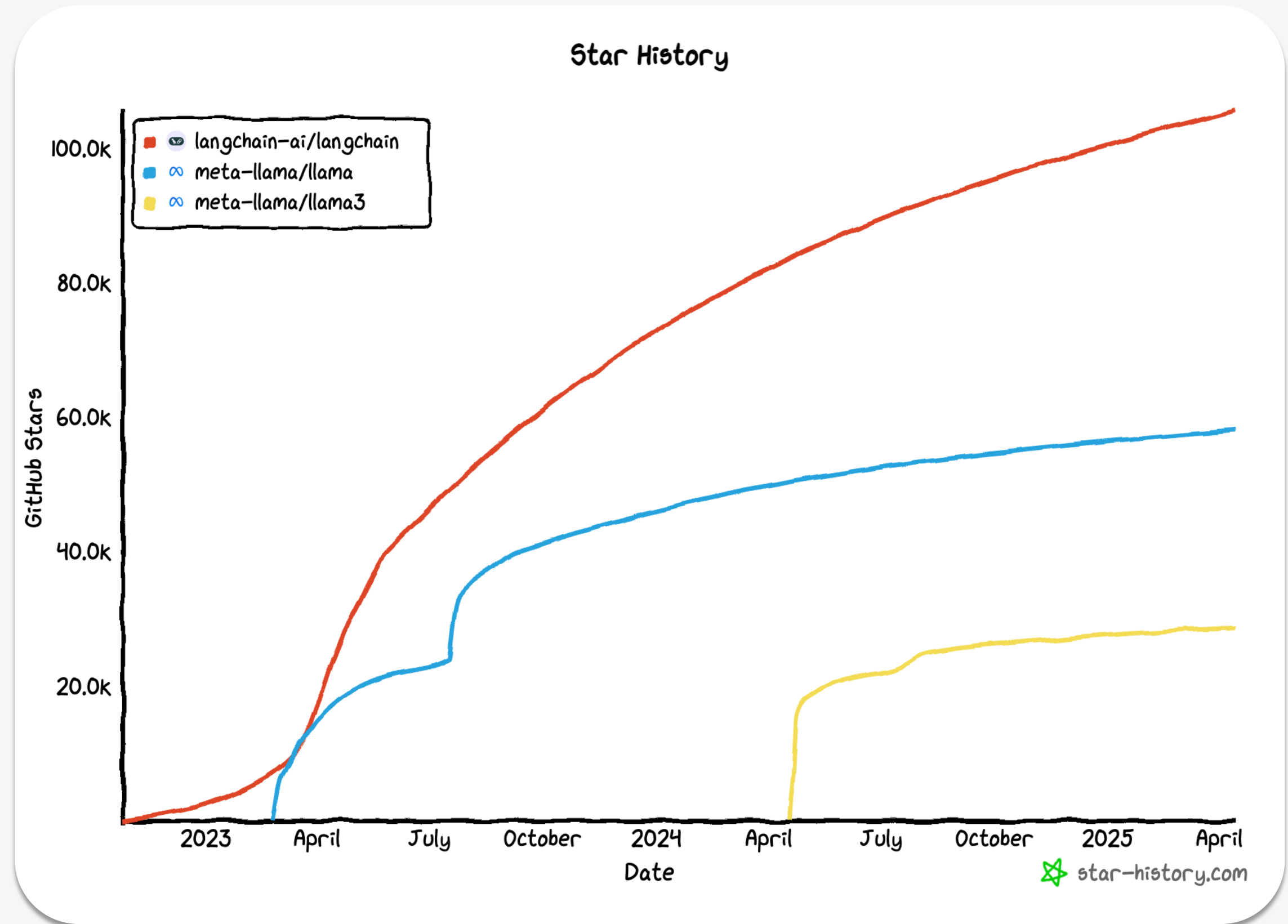


LLM 응용 툴 간 기능 비교

프레임워크	특징	차이점
LangChain	프롬프트 + 체인 + 도구 통합	복잡한 워크플로우 가능
LlamaIndex	문서 기반 질의응답에 강점	벡터 검색 특화
OpenDevin	코드 실행 중심 멀티 툴 자동화 에이전트	Git, Bash 등과 연동해 코딩 중심 작업 자동화에 특화
CrewAI	멀티에이전트 협업 프레임워크	역할 기반 Task 분담, LLM 간 협업 설계에 특화

LangChain 🦜 vs 🦙 LLaMA

- 2023년부터 2025년까지 GitHub의 Star 개수를 비교
- LangChain은 지속 가능한 커뮤니티를 기반으로 인기를 확보
- LLaMA 시리즈는 모델 공개 직후 관심이 늘지만 장기적으로 둔화하는 추세
- **LangChain** : 실용성을 중심으로 장기적인 성장을 이끌어냄
- **LLaMA** : 연구 목적에서 관심은 높지만 하드웨어가 뒷받침되어야 하므로 커뮤니티 지속성이 상대적으로 낮음
- AI 생태계에서는 단순한 모델 공개보다 **생태계를 연결하고 응용할 수 있는 툴의 가치가 부각**

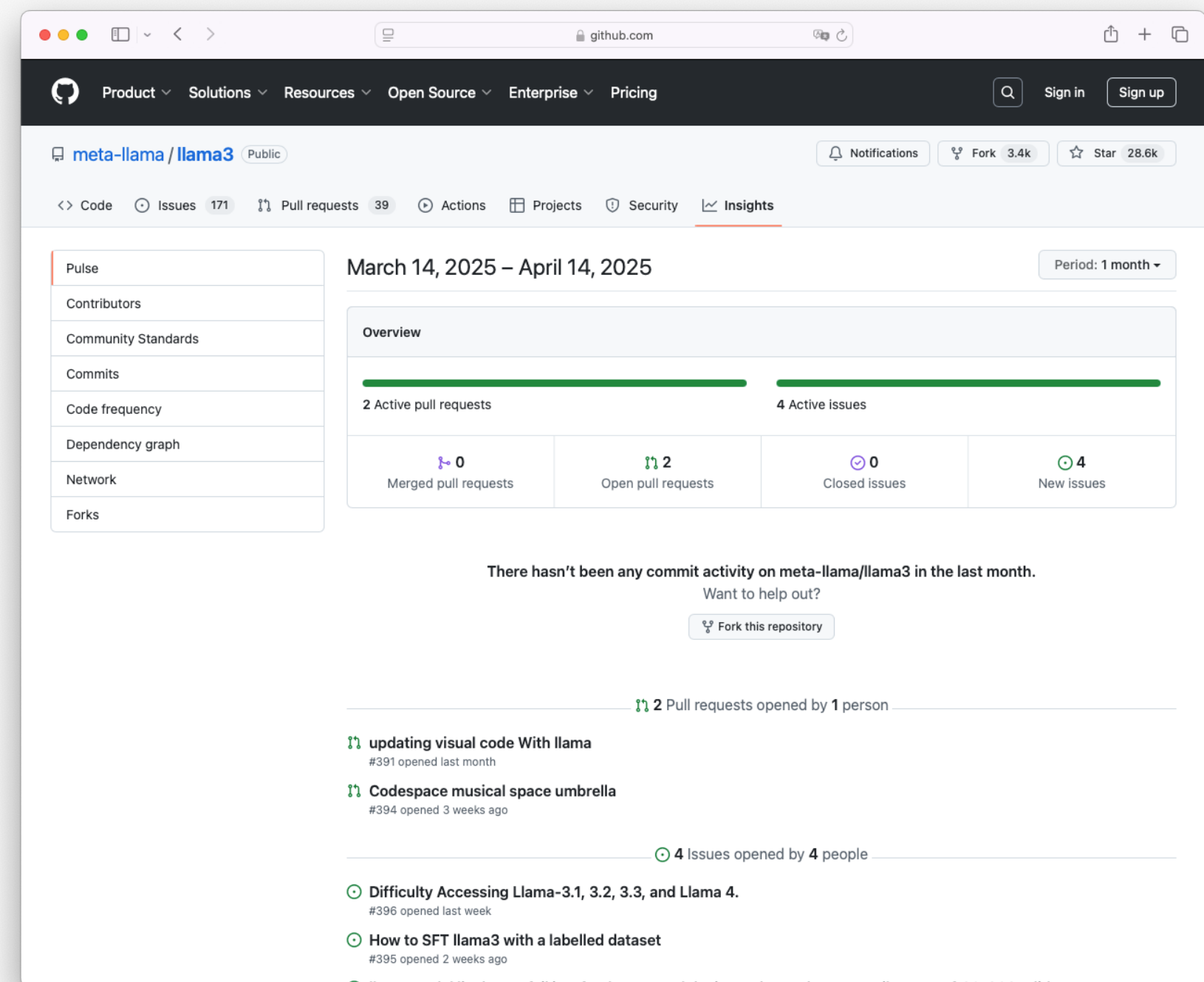
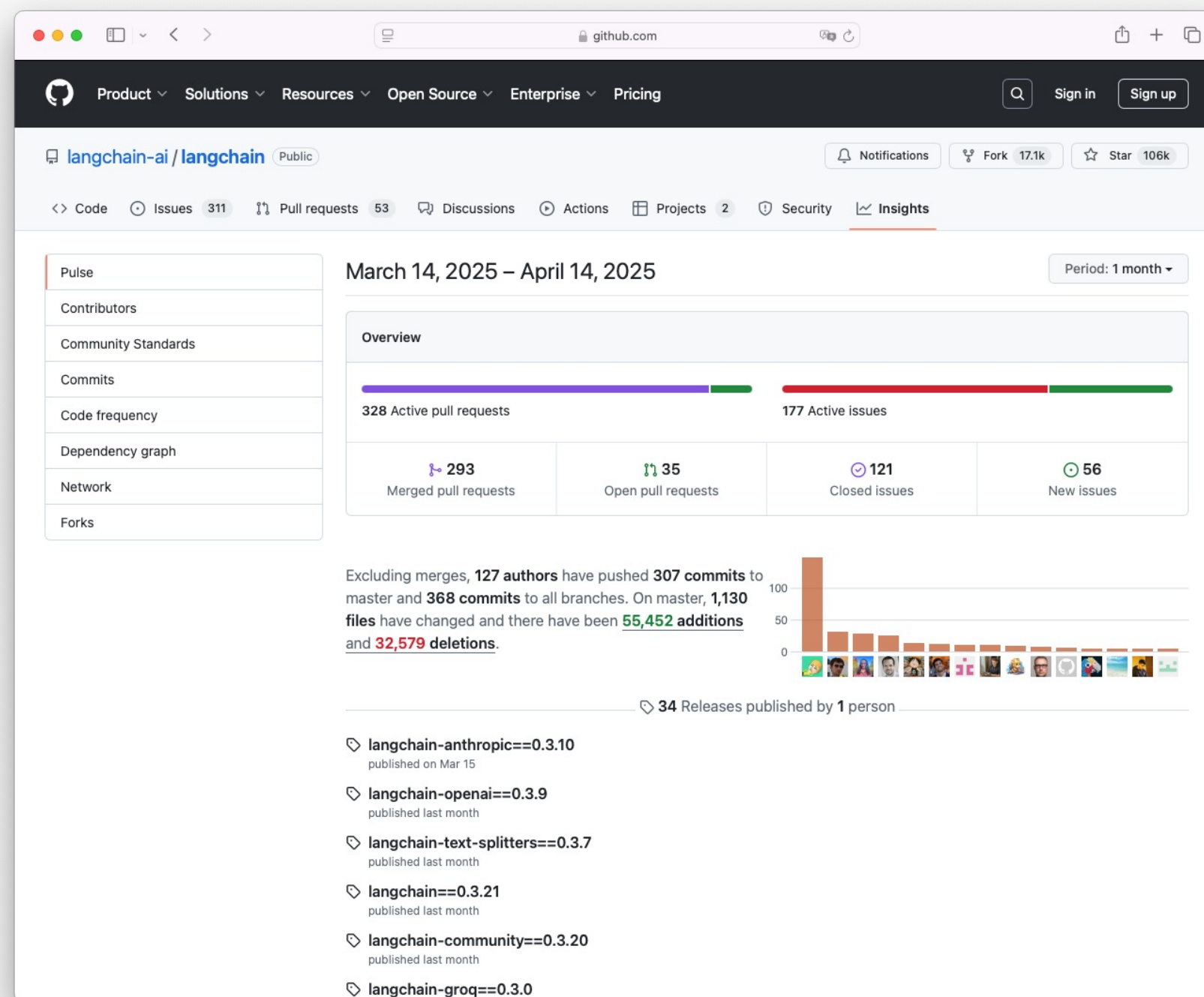




커뮤니티 활성 비교

성격이 다른 두 커뮤니티 활발한 LangChain, 정적인 LLaMA

- LangChain은 활발한 오픈소스 생태계를 기반으로 수많은 기여자가 함께 활동 중
- 실제 사용 및 유지보수, 기능 확장이 지속적으로 이루어지고 있음



LangChain : <https://github.com/langchain-ai/langchain/pulse/monthly>

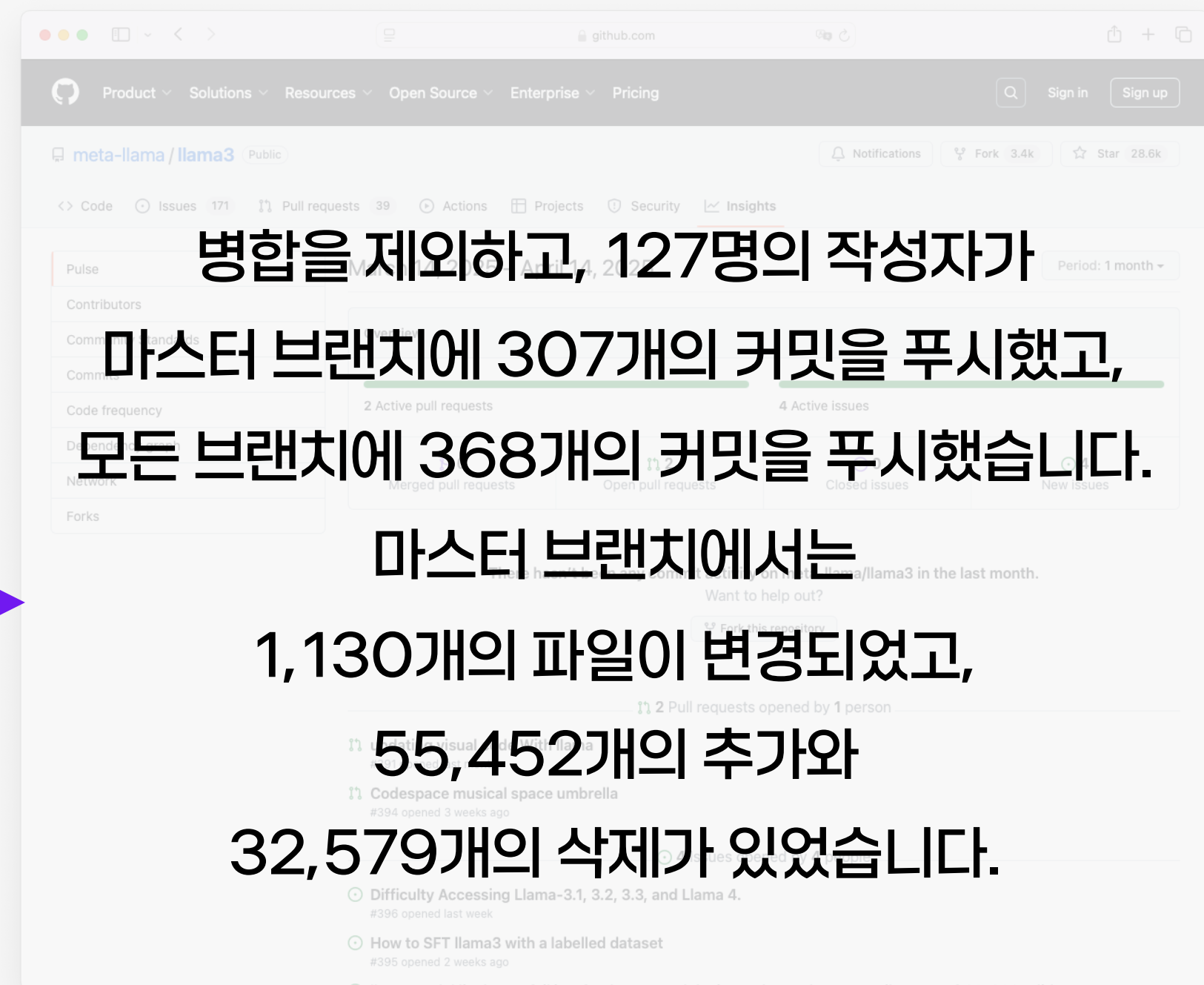
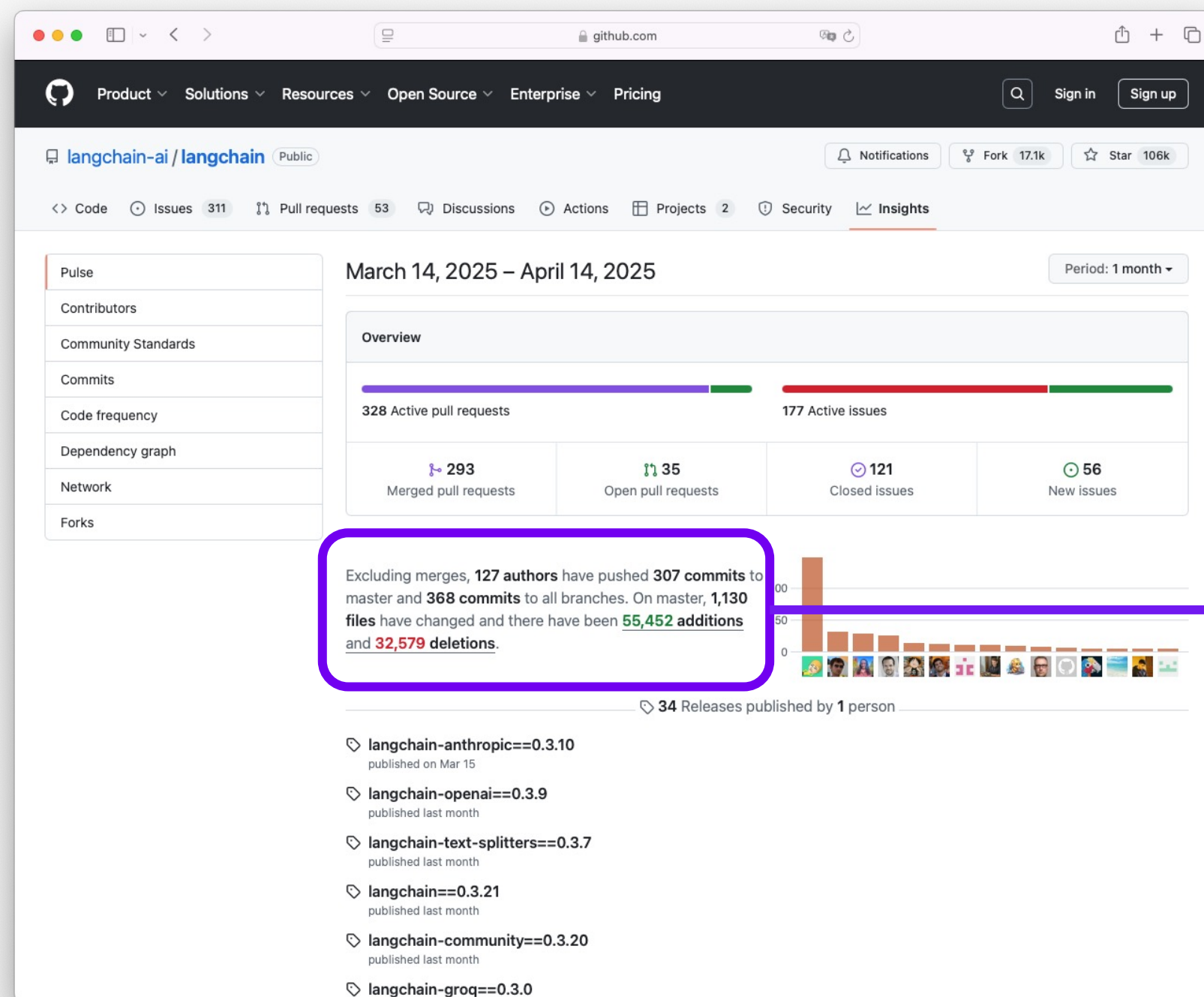
LLaMA : <https://github.com/meta-llama/llama3/pulse/monthly>



커뮤니티 활성 비교

성격이 다른 두 커뮤니티 활발한 LangChain, 정적인 LLaMA

- LangChain은 활발한 오픈소스 생태계를 기반으로 수많은 기여자가 함께 활동 중
- 실제 사용 및 유지보수, 기능 확장이 지속적으로 이루어지고 있음



LangChain : <https://github.com/langchain-ai/langchain/pulse/monthly>
LLaMA : <https://github.com/meta-llama/llama3/pulse/monthly>

2. LangChain 주요 특징



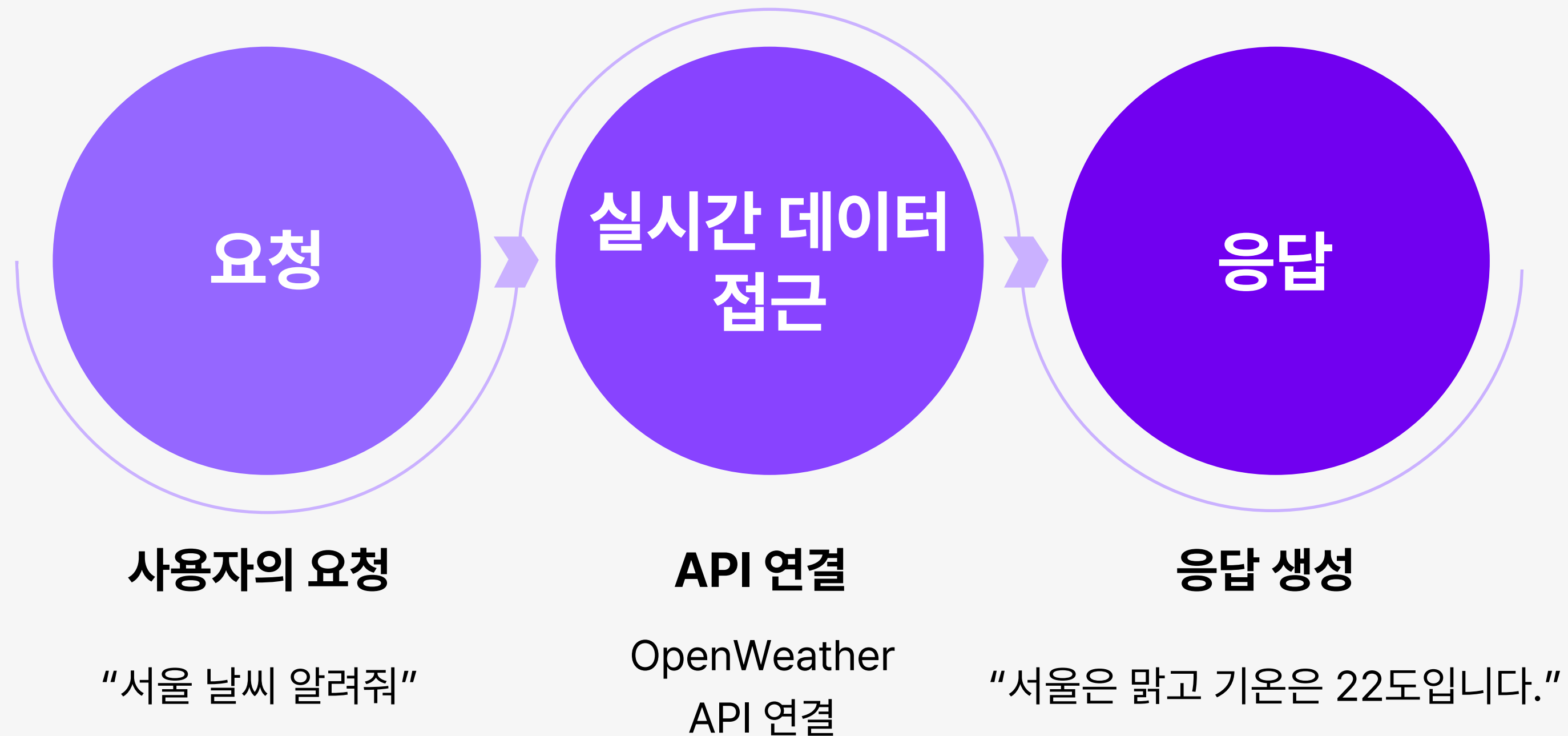
대표적인 모듈

LangChain은 다양한 기능을 모듈(블록) 단위로 나눠 설계되어 있어, 사용자가 필요에 따라 조합하거나 교체할 수 있음

	역할	사용 예시
PromptTemplate	프롬프트를 체계적으로 구성	<pre>from langchain_core.prompts import PromptTemplate template = PromptTemplate.from_template("오늘의 날씨를 요약해줘: {city}")</pre>
LLM	실제 언어모델 호출	<pre>from langchain_openai import OpenAI llm = OpenAI(model="gpt-3.5-turbo")</pre>
OutputParser	응답을 가공하거나 후처리	<pre>from langchain_core.output_parsers import StrOutputParser parser = StrOutputParser()</pre>
Chain	여러 모듈을 연결하여 흐름 구성	<pre>chain = prompt llm</pre>



LLM은 기본적으로 사전 학습된 지식만 활용 가능하지만,
LangChain은 외부 데이터 소스나 API를 연결해 실시간 정보에 접근 가능하게 함





LangChain의 Memory 기능

이전 입력, 대화 이력, 맥락 등을 저장

- 기본 LLM은 'Stateless', 즉 이전 대화 내용을 기억하지 못함
- LangChain의 Memory 기능을 통해
이전 입력, 대화 이력, 맥락 등을 저장하고 불러올 수 있음
- 대화형 시스템이나 챗봇에서는 특히 유용



MEMORY

안녕, 내 이름은 엘리스야.

반가워, 엘리스. 앞으로 잘 부탁해!
무엇을 도와줄까?

내 이름이 뭐라고?

너의 이름은 엘리스야.



다양한 언어모델과의 유연한 통합

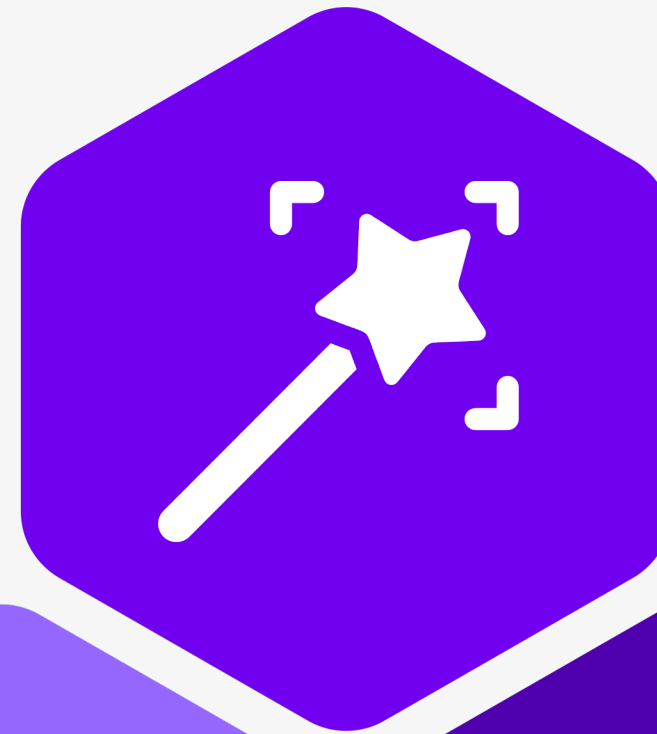
- LangChain은 다양한 LLM 백엔드를 쉽게 교체하거나 병렬로 사용할 수 있게 설계됨
 - OpenAI GPT-4, Anthropic Claude, HuggingFace Transformers, Cohere 등
- 각각의 LLM은 특성과 요금제가 다르기 때문에, 상황에 맞는 선택이 가능
 - 특정 작업은 GPT-4로, 특정 작업은 더 저렴한 모델로 분산 처리



 Claude



파이프라인처럼 구성 가능



오픈소스 기반이므로 원하는 기능을
커스터마이징하거나 기여 가능



사용자 정의 컴포넌트 추가 가능
(사용자 정의 프롬프트, 툴 등)



프로젝트, 프로덕션 앱 모두에
적합한 확장성 제공

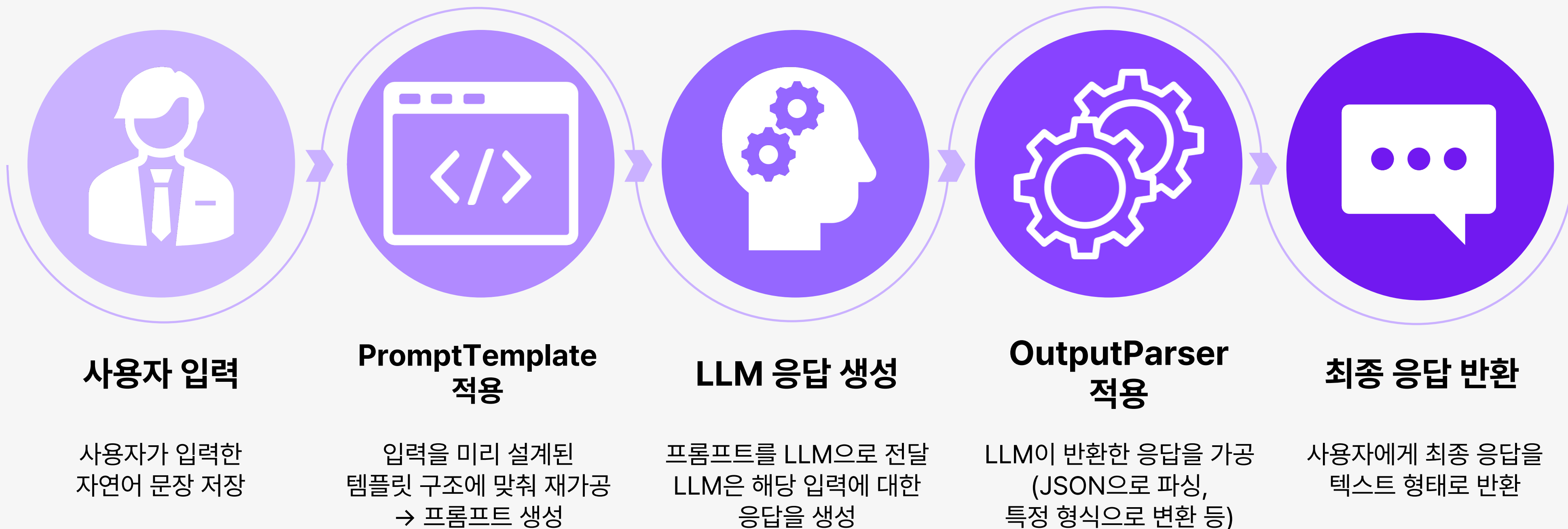
프롬프트 설계부터 외부 도구 연동, 문서 검색, 메모리 관리까지

주요 기능	설명
Prompt Engineering 지원	프롬프트를 템플릿 형태로 관리하고 변수 삽입을 통해 재사용성과 일관성 향상
Chain 설계 및 조합	여러 작업 단계를 연결해 복잡한 처리 과정을 자동화
Agent를 통한 자율적 도구 사용	LLM이 상황에 따라 적절한 도구(API, 계산기 등)를 스스로 선택해 활용
Memory를 통한 맥락 유지	이전 대화나 작업 이력을 저장해 대화형 응용에서 지속적인 맥락을 제공
Document Loader 및 Vector Store 통합	외부 문서를 불러와 임베딩하고 벡터 검색을 통해 RAG 기반 응용 가능
Toolkits 및 Plugins 사용 가능	코드 실행, 브라우징, 검색 등 다양한 기능을 플러그인 형태로 쉽게 확장



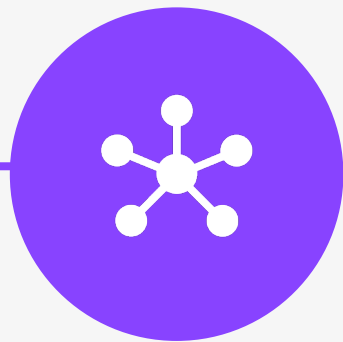
사용자 입력을 **가공**하고 응답을 **생성**하며 최종 응답을 **구성**

필요에 따라 아래 흐름에서 **Tool** 호출 등 추가



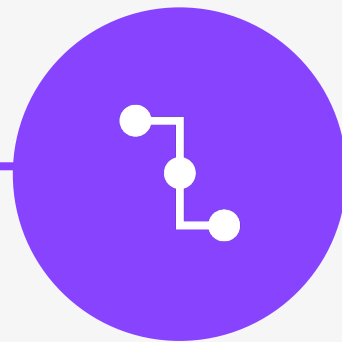
3. LangChain과 API

LangChain은 다양한 LLM과 API를 연동할 수 있는 확장성 높은 프레임워크



LLM Providers

다양한 LLM을
손쉽게 연결할 수 있음
(OpenAI, Anthropic,
Hugging Face, Cohere 등)



API/Tool 연동

LLM이 외부 툴을
직접 호출하여
복잡한 작업을 수행할 수 있음
(Google Search API,
Python REPL 등)



기타 통합 도구

Chroma, FAISS와 같은
벡터 DB부터
LangSmith같은
디버깅(추적) 도구와
연동이 쉬움